



OpsGate

DailyOps.Tech

FAQ deployment & composants V2

[MSI](#) · [licences](#) · [etapes](#) · [briques](#) · [silent](#) · [URL management](#)

[Integrateurs](#) · [DSI](#) · [pilote test](#) · [support](#)
Reponses claires et actionnables

[DailyOps.Tech](#) · [OpsGate](#) · [FR](#)



Public : intégrateur, DSI, pilote test, support

Date : 16 juillet 2026

Statut : aligné code monorepo (proxy MSI, control plane, extension)

1. Le MSI remplace-t-il le proxy du terminal ?

Oui, côté poste client / production.

Contexte	Quoi utiliser	Rôle
Dev / lab	pnpm proxy:dev (terminal)	Même code proxy, rechargement rapide
Test / prod poste	MSI opsgate-proxy-*.msi	Même proxy, Node embarqué, tâche planifiée, sans terminal

Le MSI n'est pas un autre produit : c'est le packaging Windows de packages/proxy (MITM multi-IA, soft-block / soft-mask, enroll vers l'API).

- Remplace : ouvrir un PowerShell et lancer pnpm proxy:dev sur chaque PC.
- Ne remplace pas : l'API, la console, Postgres, ni l'extension navigateur.

Install silencieux :

```
msiexec /i dist/opsgate-proxy-1.2.0.msi /qn
```

Après install : binaires sous C:/Program Files/OpsGate/Proxy/, data sous %ProgramData%/OpsGate/Proxy/.

2. Licences et « signature du code » chez le client

Deux notions différentes - ne pas les confondre.

A. Licence produit (sièges / org)

1. Le client enrôle extension et/ou proxy avec un code organisation (ex. DEMO-OPSGATE ou le code fourni).
2. L'agent joint votre tenant API (POST /v1/enroll).
3. La console assigne des sièges (licenseSeats / assignation agent).
4. Sans siège (après grâce) -> protection inactive côté agent.

Ce n'est pas la signature Authenticode du MSI.

C'est : code org + token agent + sièges gérés par le control plane.

B. Signature des packs de règles (ed25519)

Les rule packs publiés par l'API portent un checksum + signature (ed25519).

L'agent vérifie que le pack vient bien de votre control plane (anti-tamper des règles), pas le binaire MSI.

C. Signature Windows / store (hors licence OpsGate)

Signer le MSI (Authenticode) ou publier l'extension (CWS / AMO) sert à la confiance OS / navigateur (SmartScreen, force-install).

Cela n'active pas automatiquement les sièges : l'enrôlement org reste obligatoire.



En résumé : chez le client, on reconnaît l'agent par enroll (org_code + empreinte machine) et la licence par sièges org ; le pack est signé par l'API ; le MSI signé rassure Windows, pas le moteur de licences.

3. Déployer pour un test aujourd'hui (graduel)

Hypothèse : un serveur de management (votre PC ou un serveur lab) + un poste utilisateur de test.
Sur un seul PC de lab, tout peut cohabiter en 127.0.0.1.

Phase 0 - Prérequis

- Windows 10/11, droits admin pour MSI / CA
- Node 20+ et pnpm uniquement sur la machine qui build / héberge l'API
- Docker Desktop (Postgres recommandé)
- Chrome ou Edge pour l'extension

Phase 1 - Control plane (serveur de management)

```
cd C:/Users/Utilisateur/ops-gate
pnpm install

# Postgres durable (fortement recommandé)
docker compose up -d
$env:DATABASE_URL = "postgres://opsgate:opsgate@127.0.0.1:5432/opsgate"

# Terminal A - API
pnpm api:dev
# Attendu : store=postgres · http://127.0.0.1:8787
```

Vérifs :

```
curl http://127.0.0.1:8787/health
# ok: true
```

```
# Terminal B - Console
pnpm console:dev
# -> http://127.0.0.1:5173
# Login démo : admin@demo.local / 0000 (changer en prod)
```

Succès phase 1 : login console, dashboard, code org visible (ex. DEMO-OPSGATE).

Phase 2 - Extension (protection navigateur)

```
# Sur la machine de build
pnpm build:chrome
# Charger non empaquetée : chrome://extensions -> Mode développeur
# Dossier : build/chrome-mv3-prod
```

Dans l'extension Options :



1. URL API = `http://127.0.0.1:8787` (ou IP du serveur de management, ex. `http://10.0.0.12:8787`)
2. Code org = celui de la console
3. Enrôler

Succès phase 2 : agent visible console ; prompt secret sur ChatGPT -> bandeau mask/block.

Phase 3 - Proxy local (filet HTTPS) - lab monorepo

```
# Terminal C (dev)
pnpm proxy:gen-ca
certutil -addstore -user Root "packages/proxy/data/ca/ca-cert.pem"
pnpm proxy:enroll
pnpm proxy:dev
```

Phase 4 - Proxy MSI (comme le client final)

```
# Build une fois (machine de build)
pnpm proxy:msi
# -> dist/opsgate-proxy-1.2.0.msi

# Sur le poste client (admin)
msiexec /i dist/opsgate-proxy-1.2.0.msi
# ou silencieux :
msiexec /i dist/opsgate-proxy-1.2.0.msi /qn
```

Pointer l'API avant/après install si le management n'est pas en 127.0.0.1 :

```
# Exemple : variable machine (adapter selon post-install / service)
[System.Environment]::SetEnvironmentVariable(
"OPSGATE_API_URL", "http://10.0.0.12:8787", "Machine")
[System.Environment]::SetEnvironmentVariable(
"OPSGATE_ORG_CODE", "DEMO-OPSGATE", "Machine")
# Relancer le service / tâche OpsGate Proxy
```

PAC (navigateur -> proxy local seulement pour l'IA) :

```
./scripts/set-system-proxy-pac.ps1
# Off :
./scripts/set-system-proxy-pac.ps1 -Off
```

Succès phase 4 : `opsgate-proxy.cmd status` OK ; health `http://127.0.0.1:8888/opsgate-proxy/health` ; events source=proxy en console.

Phase 5 - Soft-mask on-wire (option prod)

```
$env:OPSGATE_PROXY_SOFT_MASK = "1" # rewrite body masqué
# redémarrer proxy (MSI service ou pnpm proxy:dev)
```



Ordre des commandes (récap lab mono-PC)

```
cd C:/Users/Utilisateur/ops-gate
pnpm install
docker compose up -d
$env:DATABASE_URL = "postgres://opsgate:opsgate@127.0.0.1:5432/opsgate"
pnpm api:dev # Terminal A
pnpm console:dev # Terminal B
pnpm build:chrome # puis charger l'extension
pnpm proxy:gen-ca
certutil -addstore -user Root "packages/proxy/data/ca/ca-cert.pem"
pnpm proxy:enroll
pnpm proxy:dev # Terminal C - ou MSI en phase 4
```

4. Troubleshooting (par symptôme)

Symptôme	Causes fréquentes	Actions
API « memory », tout perdu au restart	Pas de DATABASE_URL	Setter DATABASE_URL, docker compose up -d, relancer API
Extension « local only »	Pas d'enroll / mauvaise URL API	Options -> URL + code org ; firewall / CORS ; API up
Enroll refuse	Mauvais org_code	Vérifier code dans console / seed
Agent unlicensed	Pas de siège	Console -> agents / groupes -> assigner licence
Proxy ne filtre pas	PAS de PAC / MITM off / CA non trustée	set-system-proxy-pac.ps1 ; trust CA ; OPSGATE_PROXY_MITM=1
Sites IA cassés (certificat)	CA non installée	certutil -addstore -user Root .../ca-cert.pem
MSI installé mais proxy down	Tâche / post-install skip	Lancer post-install.ps1 ; opsgate-proxy.cmd status
Events proxy absents	Pas enroll proxy	OPSGATE_API_URL + pnpm proxy:enroll ou auto-enroll
Soft-mask sans effet	Env non lue / process pas redémarré	Vérifier health soft_mask ; redémarrer service
Console ne joint pas l'API	Mauvaise URL console / API down	Network tab ; curl /health

Commandes utiles :

```
curl http://127.0.0.1:8787/health
curl http://127.0.0.1:8888/opsgate-proxy/health
pnpm proxy:status
# MSI :
& "C:/Program Files/OpsGate/Proxy/opsgate-proxy.cmd" status
Get-ScheduledTask | Where-Object TaskName -like "OpsGate"
```

5. À quoi sert chaque brique ?

Brique	Rôle clair
@opsgate/engine	Cœur détection / masquage (secrets, IBAN, configs...) partagé extension + proxy
Extension (Chrome/Edge/Firefox/Safari)	UX au moment du prompt : bandeau, mask, journal, enroll



API (control plane :8787)	Enroll, policy, packs signés, events, licences, SSO/MFA/LDAP
Console MMC (:5173)	Pilotage admin : agents, policy, packs, quotas, paramètres
Postgres	Persistence multi-tenant (sinon store mémoire = lab seulement)
Proxy local (:8888)	Filet HTTPS multi-IA (MITM allowlist), observe / enforce / soft-mask
MSI Proxy	Déploiement Windows sans Node système ni terminal
PAC	N'envoie vers le proxy que les sites IA (reste du web direct)
CA locale	Certificat de confiance pour le MITM HTTPS
Rule packs	Signatures de règles versionnées poussées sans rebuild l'extension
SSO OIDC / SAML	Auth admins entreprise
MFA TOTP / WebAuthn	2e facteur admins
LDAP/AD + cron	Import users/groupes vers People
Redis (option)	Rate-limit / quotas partagés multi-instances API
Postgres RLS	Isolation SQL stricte par org_id
SIEM / metrics	Export SOC / Prometheus (selon config)
Packages store (CWS/AMO)	Distribution et force-install MDM

6. Qu'est-ce qui n'est plus le mode « V1 seul » ?

V1 n'est pas jetée : extension + API + console + Postgres restent le socle.

Ce qui change d'usage (ou n'est plus le chemin recommandé en prod poste) :

Avant (lab V1)	Maintenant (V2 prod-ready)
Proxy seulement via pnpm proxy:dev	MSI + tâche sur le poste
Un seul filet = extension	Double filet extension + proxy
Admin souvent email/mdp local	SSO / MFA / LDAP disponibles
API mono-process mémoire OK pour démo	Postgres (+ RLS) pour pilote réel
Chrome uniquement	Firefox / Safari (build) + packaging store
Block proxy brutal parfois	Soft-block 403 ou soft-mask on-wire

Toujours utilisables en lab : pnpm api:dev, console:dev, proxy:dev, store mémoire (non durable).

7. Qu'est-ce qui s'est ajouté ? Comment ça marche ?

Ajout	Comment ça marche (simple)
Proxy MITM multi-IA	Déchiffre seulement l'allowlist IA ; engine scanne le body ; observe ou coupe / masque
Soft-mask on-wire	Réécrit le body (secrets masqués) puis envoie au fournisseur IA
HTTP/2 MITM	Même logique sur flux HTTP/2 quand applicable
MSI + Node portable	Install GPO/MDM sans dépendre du Node de l'utilisateur
Démarrage silencieux	Tâche planifiée / scripts silent (pas de console noire)
SSO OIDC + SAML	Login console via IdP ; JIT admin ; option sso_enforce
MFA TOTP	Code Authenticator après login



WebAuthn	Passkeys (store process-local en l'état actuel)
LDAP cron	Sync planifiée users/groupes
Quotas / rate-limit	Limites org ; Redis optionnel si multi-API
RLS Postgres	Chaque requête SQL bridée à l'org courante
Packaging CWS / AMO / Safari	Zips policies pour stores et MDM

Flux type :

```

Utilisateur -> site IA
-> Extension (bandeau / mask)
-> [option] Proxy local (2e filet)
-> API (events, policy sync)
-> Console (visibilité DSI)

```

8. Guide utilisateur sans pnpm build / api:dev / console:dev ?

Normal. Le guide utilisateur V2 cible l'admin métier (console, extension, enrôlement), pas le développeur.

Besoin	Document / outil
Usage console & extension	GUIDE-UTILISATEUR-V2
Décideurs DSI/RSSI	DECIDEURS-V2
Build, ports, Docker, commandes	GUIDE-STACK-LOCALE, GUIDE-TEST-V2, ce FAQ
Poste final sans monorepo	MSI proxy + extension (store ou zip MDM) + API hébergée

On a toujours besoin de pnpm api:dev / console:dev / build:chrome pour :

- développer,
- héberger le control plane en lab,
- produire les artefacts (MSI, zip extension).

Ce qui les remplace côté utilisateur final :

Avant (dev)	Chez le client
pnpm api:dev	API déployée (service / serveur)
pnpm console:dev	Console buildée (console:build) servie en HTTPS
pnpm build:chrome + load unpacked	Force-install store / MDM / zip
pnpm proxy:dev	MSI + tâche silencieuse

9. Commandes silencieuses : déjà faites ?

Oui, pour le proxy Windows.

Action	Commande / mécanisme
Install MSI silencieuse	msiexec /i opsgate-proxy-*.msi /qn
Démarrage sans fenêtre	scripts/start-proxy-silent.ps1



Arrêt	start-proxy-silent.ps1 -Stop
PAC système	set-system-proxy-pac.ps1
Post-install MSI	CA + enroll + tâche planifiée (post-install.ps1)
Service / tâche	install-proxy-service-windows.ps1 (-TaskOnly ou NSSM)

Ce qui n'est pas « un seul clic silencieux pour tout le produit » : API + Postgres + console restent un déploiement serveur (Docker / services), distinct du MSI poste.

10. Peut-on modifier l'IP (URL) du serveur de management ?

Oui.

Composant	Comment changer l'URL API
Extension	Options -> champ URL API (ex. http://10.20.30.40:8787 ou HTTPS prod) puis ré-enrôler si besoin
Proxy (dev)	\$env:OPSGATE_API_URL = "http://10.20.30.40:8787" puis pnpm proxy:enroll / proxy:dev
Proxy (MSI)	Variable d'environnement machine OPSGATE_API_URL (et OPSGATE_ORG_CODE) + redémarrage tâche/service
Console	Build/config pointant vers l'API (mode avancé login / variable d'env selon déploiement)

Par défaut lab : http://127.0.0.1:8787.

En entreprise : URL stable (DNS recommandé plutôt qu'IP brute) + HTTPS.

Le proxy écoute localement (127.0.0.1:8888) : on ne change en général pas l'IP du proxy, seulement celle du control plane (management).

11. Schéma mental (une page)

[Poste utilisateur]
Extension —enroll/events—> [Serveur management]
Proxy MSI —enroll/events—> API :8787
PAC -> 127.0.0.1:8888 |
├─ Postgres
├─ Console admin
└─ Licences + packs signés

Licence = sièges org après enroll
Pack règles = signature ed25519 API
MSI signé Windows = confiance OS (≠ licence produit)

12. Fichiers utiles

Fichier	Contenu
docs/architecture/PROXY-PROD-WINDOWS.md	MSI, silent, PAC, soft-mask
docs/GUIDE-STACK-LOCALE.md	Docker, API, console, extension
docs/GUIDE-TEST-V2.md	Checklist QA + commandes
docs/ORG-CODE-AND-TENANT.md	Code org vs licences



docs/GUIDE-UTILISATEUR-V2.md

Usage admin (sans build)

docs/DECIDEURS-V2-FR.md

Argumentaire DSI

© DailyOps.Tech - OpsGate V2